CY321 Secure Software Design and Development

Secure AI Prompt Sandbox

Deliverable 1

Muhammad Afeef Bari 2023356

Mahad Aqeel 2023286

Muhammad Daniyal 2023406

1. GitHub Repository

Repository created at:
[Secure-AI-Prompt-Sandbox]

Folder Structure

secure-ai-prompt-sandbox/

```
├── Deliverable-1/   ← This report + diagrams
├── Deliverable-2/   ← Source code (initial)
├── Deliverable-3/   ← Final source + test results
├── frontend/
├── backend/
├── docs/            ← Architecture + threat model diagrams
└── README.md
```

2. Security Requirements and Planning

The Secure AI Prompt Sandbox is designed to provide a protective middleware layer between users and Large Language Model (LLM) APIs. Because LLMs process natural language inputs that may contain malicious instructions, the system must enforce strong security controls to ensure that user prompts cannot manipulate system behavior or expose sensitive information. The overall security goal is to maintain the confidentiality of system prompts and credentials, the integrity of prompt processing, and the availability of the service while still allowing legitimate user interaction with the model.

A key requirement of the system is the **implementation of strict access control mechanisms**. Users must authenticate before interacting with the platform, and the system must differentiate between standard users and administrators through Role-Based Access Control (RBAC). Administrative users will have permissions to configure security policies, review logs, and manage system settings, while standard users will only be able to submit prompts and view responses. This separation ensures that sensitive administrative functions cannot be accessed by unauthorized individuals.

Another critical requirement is the **implementation of a Prompt Validation Engine** that evaluates all incoming user prompts before they are forwarded to the LLM API. Because prompt injection attacks often attempt to override system instructions or manipulate the model's behavior, the validation engine must analyze prompts for suspicious patterns such as instruction override attempts, role manipulation, multilingual bypass strategies, and indirect prompt injections embedded within user inputs. Prompts identified as potentially malicious will be flagged and assigned a risk score, allowing the system to determine whether the prompt should be blocked, modified, or allowed.

Finally, the project emphasizes **usable security principles**. When a prompt is flagged as suspicious, the system should provide clear and understandable feedback rather than silently rejecting the request. The system will display a visual risk indicator (e.g., low, medium, or high risk) and provide guidance on how the prompt can be reformulated safely. This approach helps users understand security policies while maintaining system protection.

ID	Category	Requirement	Priority	Source Component
SR-01	Authentication	All users must authenticate before submitting prompts or accessing any API endpoint.	C	RBAC
SR-02	Authorization	The system enforces RBAC distinguishing standard users from administrators.	C	RBAC
SR-03	Prompt Validation	All prompts must be analyzed for suspicious patterns, instruction overrides, and adversarial structures before forwarding.	C	Validation Engine
SR-04	Injection Detection	The system must detect sandwich attacks, role manipulation, indirect injection, attention blink exploitation, and multilingual injection attempts.	C	Validation Engine
SR-05	Context Isolation	System-level instructions must be protected and cannot be overridden or exposed via user input.	C	Context Layer
SR-06	Policy Enforcement	All prompts must be evaluated against predefined security rules; outcome must be allowed, modified, or blocked.	C	Policy Module
SR-07	API Key Security	LLM API credentials must be stored in an encrypted vault and must never be accessible from the client side.	C	API Key Vault
SR-08	Audit Logging	All prompt metadata, system decisions, and security events must be logged with hash-chained entries for tamper-evidence.	H	Audit Logging
SR-09	Rate Limiting	The system must enforce rate limits per user to mitigate denial-of-service and repeated policy violation attempts.	H	Rate Limiter
SR-10	Transport Security	All communication between frontend, backend, and LLM API must use HTTPS/TLS.	C	All Layers
SR-11	Risk Feedback	The system must display a real-time risk score (Low/Medium/High) and a human-readable reason when a prompt is flagged or blocked.	M	Usable Security
SR-12	Safe Reformulation	When a prompt is blocked, the system must offer guidance for safe reformulation rather than silently rejecting.	M	Usable Security

Priority Key: C = Critical H = High M = Medium

3. Risk Management

#	Risk	Component	Likelihood	Impact	Mitigation
R-01	Prompt injection bypasses validation engine	Validation Engine	High	High	Multi-pattern detection: sandwich, role manipulation, indirect injection, multilingual
R-02	System prompt exposed via poor context management	Context Layer	Medium	High	Strict separation of system instructions and user input; context isolation per session
R-03	LLM API key leaked client-side or via error	API Key Vault	Medium	High	AES-256 encrypted vault; keys never sent to frontend; sanitized error messages
R-04	Audit log tampered to erase abuse evidence	Audit Logging	Low	High	SHA-256 hash chaining: each entry contains hash of previous entry
R-05	RBAC bypass via forged or replayed token	RBAC / Auth	Medium	High	Short-expiry JWT; server-side role validation on every endpoint; no client trust
R-06	Rate limit evasion causes DoS or credit exhaustion	Rate Limiter	High	Medium	Sliding window counter per user and per IP
R-07	Cross-session context leakage between users	Context Layer	Low	High	UUID-bound session objects; cleared on session end
R-08	Multilingual input bypasses injection heuristics	Validation Engine	Medium	High	Cross-lingual prompt analysis module; alignment gap detection
R-09	XSS via reflected prompt content in UI	Frontend	Medium	Medium	React auto-escaping; strict Content Security Policy headers

4. Threat Modeling — STRIDE

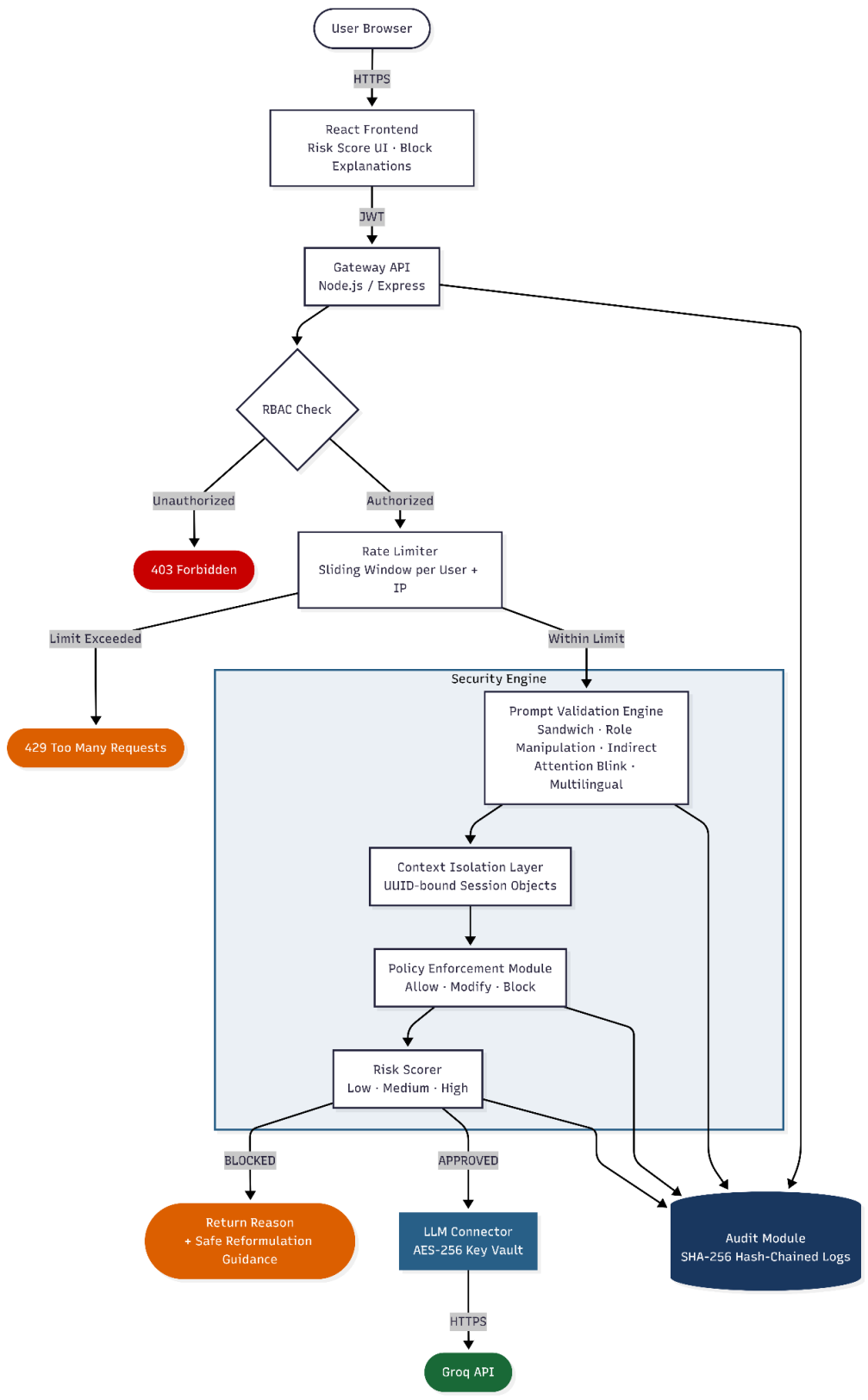
STRIDE is applied across all five system components as identified in the proposal: Frontend, Gateway API, Security Engine (Validation + Policy + Context), Audit Module, and LLM Connector.

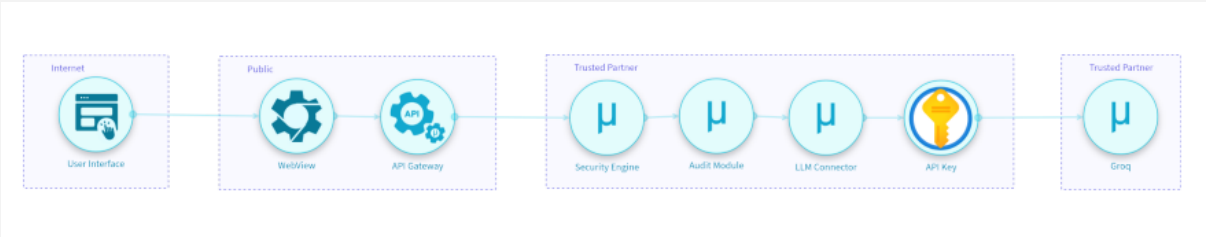
STRIDE	Component	Attack Scenario	Likelihood	Impact	Control
Spoofing	Auth / RBAC	Attacker forges JWT to impersonate admin user	Medium	High	Short-expiry JWT; server-side role check
Spoofing	LLM Connector	Attacker spoofs LLM API endpoint to intercept responses	Low	High	Hardcoded, validated endpoint URLs
Tampering	Validation Engine	Prompt injection overrides system instructions (sandwich / role manipulation)	High	High	Multi-layer injection detection heuristics
Tampering	Audit Module	Admin modifies log entries to remove evidence of abuse	Low	High	SHA-256 hash chaining per log entry
Repudiation	Gateway API	User denies sending a malicious or policy-violating prompt	Medium	Medium	Tamper-evident log: user ID + timestamp + prompt hash
Info Disclosure	API Key Vault	LLM API key exposed via error message or client-side leak	Medium	High	AES-256 vault; sanitized errors; no key in frontend
Info Disclosure	Context Layer	User A reads User B session context via request manipulation	Low	High	Session-scoped isolation; UUID-bound objects
Info Disclosure	Context Layer	System prompt leaked indirectly through poor context management	Medium	High	Strict system/user instruction separation
DoS	Gateway API	Bot floods endpoint exhausting LLM API credits	High	Medium	Sliding window rate limiter per user + IP
DoS	Validation Engine	Crafted long prompt causes ReDoS via regex backtracking	Medium	Medium	Max prompt length; safe regex patterns; timeout
EoP	RBAC Module	Standard user accesses admin endpoints via direct URL manipulation	Medium	High	Server-side role check on every endpoint
EoP	Validation Engine	Jailbreak prompt tricks LLM into behaving as unrestricted model	High	High	Jailbreak heuristics; system prompt lock; output filtering

5. System Architecture

Three primary components as defined in the proposal: Frontend UI, Secure Backend Gateway, and Controlled LLM API Interface. The backend is the core security layer — no direct user-to-LLM communication exists.

Data Flow



Trust Boundaries

- User ↔ Frontend: HTTPS only; no sensitive data in URL parameters
- Frontend ↔ Gateway API: JWT authentication on every request
- Gateway ↔ Security Engine: only authenticated, rate-checked requests reach here
- Security Engine ↔ LLM Connector: only approved, sanitized prompts pass this boundary
- LLM Connector ↔ LLM Provider: encrypted channel; API key never leaves server